

Downloading and Running an Open-Source LLM on Monash M3

A beginner's step-by-step walkthrough — from first SSH login to generating text with a local model

Written for: new M3 users on the al49 project (Trustworthiness Oracle for AI-based systems)

What you'll do

This guide walks through the whole process of running an open-source large language model (LLM) on Monash's M3 HPC cluster, starting from nothing. By the end, you'll have connected to M3, requested a GPU, set up an isolated Python environment, installed PyTorch and Hugging Face's libraries, and generated text from a real model running on a Tesla T4 GPU.

The steps are:

1. Connect to M3 over SSH
2. Request an interactive GPU session (a compute node) via smux
3. Load the Python/CUDA modules you need
4. Create an isolated Python environment in your project's scratch space
5. Install PyTorch, Transformers, and Accelerate
6. Write and run a script that downloads and runs a model
7. Know what to do next time you log back in

Why this matters: M3's login nodes are shared by everyone and are only meant for light housekeeping (editing files, submitting jobs). Downloading multi-gigabyte model weights or running inference on a login node is against the rules and will be heavy on other users — always do this work on a compute node.

Before you start

You'll need:

- An active M3 account and password (set at password-hpc.erc.monash.edu.au)
- Your M3 username (e.g. wlee0060)
- A project account code to charge compute time to (e.g. al49) — this comes in your project activation email
- A terminal with SSH access

Step 1 — Connect to M3

From your own machine's terminal, SSH into the login node:

```
ssh <your-username>@m3.massive.org.au
```

Enter your M3 cluster password when prompted (this is separate from your Monash account password). You'll land on a login node, e.g. m3-login2 — you'll see this reflected in your terminal prompt.

Step 2 — Request a GPU compute node

Never run downloads or model inference directly on the login node. Instead, request an interactive session on a compute node with a GPU attached, using the smux command:

```
smux new-session -p gpu --gres=gpu:T4:1 -t 02:00:00 --mem=32G --cpuspertask=4 -A  
<your-project-code>
```

What each flag means

Flag	Meaning
-p gpu	Partition to use — "gpu" gives you access to T4 or A40 GPUs
--gres=gpu:T4:1	Request 1 Tesla T4 GPU specifically
-t 02:00:00	Time limit for the session (2 hours here)
--mem=32G	RAM to allocate
--cpuspertask=4	CPU cores to allocate
-A <project-code>	Which project's compute allocation to charge, e.g. al49

Common mistake: smux uses its own flag names, not the standard sbatch ones. It's `--cpuspertask` (no dashes), not `--cpus-per-task`. If you see "unrecognized arguments", check the flag name with `smux new-session --help`.

Once the session starts, your prompt will change to a compute node name (e.g. m3t100 instead of m3-login2) — that's your confirmation you're on a compute node with a GPU attached.

Confirm the GPU is visible:

```
nvidia-smi
```

You should see your GPU (e.g. Tesla T4) listed with its memory and driver/CUDA version.

Step 3 — Load Python and CUDA modules

M3 provides software through the module system rather than having everything installed by default. Check what's available:

```
module avail python  
module avail anaconda  
module avail cuda
```

If `module avail python` returns nothing, look for `anaconda` instead — it bundles Python with `conda` and `pip`. Load whichever version you find, along with a CUDA module:

```
module load anaconda/2024.02-1  
module load cuda/12.8
```

Confirm both loaded correctly:

```
python3 --version  
which conda
```

Tip: Module names and versions vary between clusters and change over time. Always check with `module avail` rather than assuming the exact version string — copy the version that's actually listed.

Step 4 — Create an isolated Python environment

Create your environment inside your project's scratch space, not your home directory — model weights and packages add up to many gigabytes, and home directories have small quotas.

Recommended: [Python venv](#)

venv is simple and reliable on shared HPC systems. (conda is also available, but its default solver can be misconfigured on some clusters — if you hit solver errors, switch to venv as shown here.)

```
python3 -m venv /home/<username>/<project-code>_scratch/llm-env
source /home/<username>/<project-code>_scratch/llm-env/bin/activate
pip install --upgrade pip
```

Once activated, your prompt will be prefixed with (llm-env) — that confirms you're working inside the isolated environment and won't affect the shared system Python.

If you use conda instead

Conda needs its shell hook sourced before activate will work, and its default solver can misbehave. If you see "CondaValueError: ... libmamba ... not recognized", add --solver=classic:

```
source /apps/anaconda/2024.02-1/etc/profile.d/conda.sh
conda create -p /home/<username>/<project-code>_scratch/llm-env python=3.11 -y
--solver=classic
conda activate /home/<username>/<project-code>_scratch/llm-env
```

Step 5 — Install PyTorch and Hugging Face libraries

With your environment activated, install PyTorch built for CUDA, plus the Hugging Face libraries for downloading and running models:

```
pip install torch --index-url https://download.pytorch.org/whl/cu121
pip install transformers accelerate huggingface_hub
```

This downloads a couple of gigabytes, so it can take a few minutes. Once finished, confirm PyTorch can see your GPU:

```
python3 -c "import torch; print(torch.__version__, torch.cuda.is_available(),
torch.cuda.get_device_name(0))"
```

Expected output looks like: 2.5.1+cu121 True Tesla T4 — the True confirms PyTorch has GPU access.

Step 6 — Download and run a model

A Tesla T4 has 15GB of GPU memory, which comfortably fits a small-to-medium model (around 3–8 billion parameters) at reduced precision. For a first run, a smaller instruction-tuned model such as Qwen2.5-3B-Instruct is fast to download and quick to test.

Create a script:

```
cat > /home/<username>/<project-code>_scratch/run_llm.py << 'EOF'
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

model_name = "Qwen/Qwen2.5-3B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    dtype=torch.bfloat16,
    device_map="auto"
)
```

```
prompt = "Explain what a trustworthiness oracle for AI systems might do, in two sentences."
messages = [{"role": "user", "content": prompt}]
inputs = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    return_tensors="pt",
    return_dict=True
).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=150, do_sample=True, temperature=0.7)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
EOF
```

Version note: Newer versions of the transformers library expect `apply_chat_template(..., return_dict=True)` and `model.generate(**inputs, ...)` rather than passing the tokenizer output straight in as a single tensor. If you see an `AttributeError` mentioning 'shape', this is almost always the cause.

Point the model cache at your scratch space (so downloaded weights don't fill up your home quota), then run the script:

```
export HF_HOME=/home/<username>/<project-code>_scratch/hf-cache
python3 /home/<username>/<project-code>_scratch/run_llm.py
```

The first run downloads the model (several GB, a minute or two on M3's network) and then prints the generated response. Subsequent runs reuse the cached weights and start much faster.

Other models you can try

To try a different model, just change the `model_name` string in the script to any model's Hugging Face repo ID — everything else stays the same. A Tesla T4 has 15GB of GPU memory, so the table below notes what fits comfortably at reduced precision (bfloat16/float16) versus what needs 4-bit quantization to fit.

Model (Hugging Face repo ID)	Size	Notes
Qwen/Qwen2.5-3B-Instruct	3B	Fits easily, fast to download — good default for testing
microsoft/Phi-3-mini-4k-instruct	3.8B	Small and strong for its size, fits easily
google/gemma-2-9b-it	9B	Fits in bfloat16 on a T4, but leaves little headroom
mistralai/Mistral-7B-Instruct-v0.3	7B	Fits in bfloat16; gated — requires accepting the license on Hugging Face
meta-llama/Llama-3.1-8B-Instruct	8B	Fits in bfloat16; gated — requires accepting the license and an HF token
Qwen/Qwen2.5-14B-Instruct	14B	Too large for a T4 at full precision — needs 4-bit quantization

Gated models (Llama, Mistral, etc.)

Some model publishers require you to accept a license on the model's Hugging Face page before you can download it, and require an access token. If you get a 401/403 error trying to load one of these:

1. Visit the model's page on huggingface.co and accept the license (you'll need a free Hugging Face account).
2. Create an access token at huggingface.co/settings/tokens.
3. Log in from the compute node with the token.

```
huggingface-cli login
```

Paste your token when prompted. After that, gated models download the same way as any other model.

If a model doesn't fit — 4-bit quantization

For larger models (roughly 13B+ parameters), install bitsandbytes and load the model in 4-bit to fit within the T4's memory:

```
pip install bitsandbytes
```

Then in your script, replace the `from_pretrained` call with:

```
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    load_in_4bit=True,
    device_map="auto"
)
```

Logging back in next time

Your environment and downloaded model persist in scratch space, but each new session needs the modules loaded and environment activated again:

```
ssh <your-username>@m3.massive.org.au
smux new-session -p gpu --gres=gpu:T4:1 -t 02:00:00 --mem=32G --cpuspertask=4 -A
<project-code>
module load anaconda/2024.02-1
source /home/<username>/<project-code>_scratch/llm-env/bin/activate
export HF_HOME=/home/<username>/<project-code>_scratch/hf-cache
```

Once activated, you can rerun your script or write a new one directly.

Troubleshooting quick reference

Symptom	Fix
smux: error: unrecognized arguments: --cpus-per-task	Use --cpuspertask (no dashes) — smux has its own flag names, different from sbatch.
CondaValueError: ...libmamba... not recognized	Add --solver=classic to conda create, or switch to python3 -m venv instead.
CondaError: Run 'conda init' before 'conda activate'	Source the shell hook first: source /apps/anaconda/<version>/etc/profile.d/conda.sh
AttributeError ... 'shape' during model.generate()	Use return_dict=True in apply_chat_template and call model.generate(**inputs, ...) instead of model.generate(inputs, ...).
No CUDA device found / torch.cuda.is_available() is False	You're likely still on a login node. Check your prompt shows a compute node name, and confirm with nvidia-smi.
Job stuck waiting to start	GPU partitions can queue depending on cluster load. Check status with squeue -u <username> in a second terminal.

Useful commands — viewing and navigating your files

Standard Linux commands work the same on M3 as anywhere else. A few that are handy for checking on your project folders and downloaded model files:

```
ls                # list files in the current folder
ls -la            # include hidden files, with permissions/sizes/dates
```

```
ls -lh          # human-readable file sizes (e.g. 1.2G instead of bytes)
ls -R          # recursive listing, including subfolders
```

To check your project folders specifically:

```
ls -la /home/<username>
ls -la /home/<username>/<project-code>
ls -la /home/<username>/<project-code>_scratch
```

For a visual folder tree instead of a flat list:

```
tree /home/<username>/<project-code>_scratch
```

To check how much storage you're using against your quota in each folder, M3 provides a dedicated command:

```
user_info
```

Good practice on M3

- Do all downloads, environment setup, and model inference on a compute node — never the login node.
- Store environments and model weights in your project's scratch folder, not \$HOME (home quotas are small).
- Use the dedicated data transfer node (m3-dtn.massive.org.au) with scp/rsync for large file transfers, not the login node.
- Once you've confirmed a workflow interactively, consider wrapping it in an sbatch batch script so it runs unattended and frees up your session.
- Check `squeue -u <username>` to see your running/queued jobs, and `scancel <jobid>` to cancel one you no longer need.